

8 位动态扫描设计

1、实验任务：

- 1.1 学习硬件扫描显示电路的设计。
- 1.2 掌握利用 FPGA 设计 8 位七段数码管扫描显示驱动电路的方法。

2、实验原理：

关于数码管原理及驱动方式，可查找相关资料。LED 有段码和位码之分，所谓段码就是让 LED 显示出“8.”的 8 位数据，一般情况下要通过一个译码电路，将输入的 4 位 2 进制数转换为与 LED 显示对应的 6 位段码。位码也就是 LED 的显示使能端，对于共阳级的 LED 而言，低电平使能。要让 8 个 LED 同时工作，显示数据，就是要不停的循环扫描每一个 LED，并在使能每一个 LED 的同时，输入所需显示的数据对应的 8 位段码。虽然 8 个 LED 是依次显示，但是受视觉分辨率的影响，看到的现象是 8 个 LED 同时工作。

多个数码管动态扫描显示，是将所有数码管的相同段并联在一起，通过选通信号分时控制各个数码管的公共端，循环点亮多个数码管，并利用人眼的视觉暂留现象，只要扫描的频率大于 50Hz，将看不到闪烁现象。

HX4S20 板采用 8 位共阳数码管，可参照实验 4，或原理图查找“使用手册”，图 1.1 所示的是 8 位数码扫描显示电路，其中每个数码管的 8 个段 h、g、f、e、d、c、b、a（h 是小数点）都分别连在一起，8 个数码管分别由 8 个选通信号 k1~k8 来选择。被选通的数码管显示数据，其余关闭。如在某一时刻，k3 为低电平，其余选通信号为高电平，这时仅 k3 对应的数码管显示来自段信号端的数据，而其他 7 个数码管呈现关闭状态。根据这种电路状况，如果希望在 8 个数码管显示希望的数据，就必须使得 8 个选通信号 k1~k8 分别被单独选通，同时在段信号输入口加上希望该对应数码管上显示的数据，于是随着选通信号的扫变，就能实现扫描显示的目的。



1.1 8 位数码实物图

3、实验程序：

//实现的动态数码管显示,可在第 3 个 always 里修改需要显示的数字

```

module sm_scan(
    clk,
    rst,
    dataout,
    en
);
input clk,rst;
output[6:0] dataout;
output[7:0] en;//COM 使能输出
reg[6:0] dataout;//各段数据输出
reg[7:0] en;
reg[15:0] cnt_scan;//扫描频率计数器
reg[3:0] dataout_buf;
always@(posedge clk or negedge  rst)
    begin
        if(!rst)
            begin //低电平复位
                cnt_scan<=0;
            end
        else
            begin
                cnt_scan<=cnt_scan+1'b1;
            end
        end
    end
always @(cnt_scan)//段码扫描频率
    begin
        case(cnt_scan[15:13])
            3'b000 :    en = 8'b1111_1110;
            3'b001 :    en = 8'b1111_1101;
            3'b010 :    en = 8'b1111_1011;
            3'b011 :    en = 8'b1111_0111;
            3'b100 :    en = 8'b1110_1111;
            3'b101 :    en = 8'b1101_1111;
            3'b110 :    en = 8'b1011_1111;
            3'b111 :    en = 8'b0111_1111;
            default :    en = 8'b1111_1110;
        endcase
    end
end

```

```

always@(en) //对应 COM 信号给出各段数据，段码
begin
    case(en)
        8'b1111_1110:      dataout_buf=1;//输入将要显示的数字
        8'b1111_1101:      dataout_buf=2;
        8'b1111_1011:      dataout_buf=3;
        8'b1111_0111:      dataout_buf=4;
        8'b1110_1111:      dataout_buf=5;
        8'b1101_1111:      dataout_buf=6;
        8'b1011_1111:      dataout_buf=7;
        8'b0111_1111:      dataout_buf=8;
        default:            dataout_buf=0;
    endcase
end
always@(dataout_buf)
begin
    case(dataout_buf) //将要显示的数字译成段码
        4'b0000 :    dataout <= 7'b1000_000;//7'b0111_111;
        4'b0001 :    dataout <= 7'b1111_001;//7'b0000_110;
        4'b0010 :    dataout <= 7'b0100_100;//7'b1011_011;
        4'b0011 :    dataout <= 7'b0110_000;//7'b1001_111;
        4'b0100 :    dataout <= 7'b0011_001;//7'b1100_110;
        4'b0101 :    dataout <= 7'b0010_010;//7'b1101_101;
        4'b0110 :    dataout <= 7'b0000_010;//7'b1111_101;
        4'b0111 :    dataout <= 7'b1111_000;//7'b0000_111;
        4'b1000 :    dataout <= 7'b0000_000;//7'b1111_111;
        4'b1001 :    dataout <= 7'b0010_000;//7'b1101_111;
        4'b1010 :    dataout <= 7'b0001_000;//7'b1110_111;
        4'b1011 :    dataout <= 7'b0000_011;//7'b1111_100;
        4'b1100 :    dataout <= 7'b1000_110;//7'b0111_001;
        4'b1101 :    dataout <= 7'b0100_001;//7'b1011_110;
        4'b1110 :    dataout <= 7'b0000_110;//7'b1111_001;
        4'b1111 :    dataout <= 7'b0001_110;//7'b1110_001;
        default :    dataout <= 7'b1000_000;//7'b0111_111;
    endcase
end

```

```

        endcase
    end
endmodule

```

4、引脚约束：

```

set_pin_assignment { clk } { LOCATION = R7; IOSTANDARD = LVCMOS33; PULLTYPE =
PULLUP; }
set_pin_assignment { dataout[0] } { LOCATION = C15; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { dataout[1] } { LOCATION = C16; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { dataout[2] } { LOCATION = B6; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { dataout[3] } { LOCATION = A5; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { dataout[4] } { LOCATION = B5; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { dataout[5] } { LOCATION = A8; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { dataout[6] } { LOCATION = A7; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { en[0] } { LOCATION = D11; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { en[1] } { LOCATION = E11; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { en[2] } { LOCATION = F10; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { en[3] } { LOCATION = C13; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { en[4] } { LOCATION = E10; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { en[5] } { LOCATION = E12; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { en[6] } { LOCATION = E16; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { en[7] } { LOCATION = F16; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }

```

```
set_pin_assignment { rst } { LOCATION = A2; IOSTANDARD = LVCMOS33; PULLTYPE =  
PULLUP; }
```

5、下载测试：

